Barrera automática de almacén

Guía de montaje, mediciones y actividades | Un proyecto compartido

Una caja se acerca, el sensor la detecta y el servomotor levanta una barrera de cartón. Los LED muestran la orden de apertura. La computadora registra las detecciones. Los alumnos construyen y comprueban cada parte antes de reunirlas.

Qué construiremos

Entrada	Decisión en Arduino	Salida
Distancia del objeto y botón de bloqueo	Abrir si hay presencia y no hay bloqueo. Esperar antes de cerrar.	Servo: posición del brazo. LED: estado ordenado. Monitor: conteo.

Material electrónico: únicamente piezas previstas en el ELEGOO UNO R3 Super Starter Kit. Base y brazo de cartón; palitos opcionales. Computadora y multímetro MUT-830 del profesor. No requiere comprar piezas ni usar internet durante la práctica.

Un alcance claro

Es una maqueta didáctica de mesa. Un solo sensor cuenta detecciones, no confirma cruces ni protege personas. El LED indica la orden del programa, no demuestra que el brazo llegó a su posición. Arduino permite estudiar entradas, lógica y salidas; no es un PLC industrial.

Relaciona circuito, voltaje, corriente, resistencia, potencia, LED, motor, sensor, lógica, tiempo y conteo. Transformadores, CA, contactores y circuitos de potencia se explican por comparación; no se montan aquí. El servo ya incorpora motor, engranes y electrónica de control.

Cómo usar esta guía

Páginas 2-3: piezas y preparación. 4-6: conexiones y alimentación. 7-8: pruebas y construcción. 9: mediciones. 10-11: actividades, respuestas y fallas. 12: fuentes. Desde la 13: programa completo.

Estado de validación: conexiones y lógica revisadas documentalmente; programa comprobado con pruebas de lógica en computadora. El montaje físico queda por probar cuando llegue el kit. La pila de 9 V puede limitar la duración o el movimiento: la guía incluye un modo sin servo.

1. Separar las piezas

No hace falta utilizar todo el kit

Cant.	Pieza	Uso
1	UNO R3 y cable USB	Programa y alimentación de la lógica
1	Protoboard y cables Dupont	Conexiones desmontables
1	HC-SR04 ultrasónico	Detectar una caja cercana
1	SG90, brazo plástico y tornillo	Mover una barrera ligera
2	LED: rojo y verde	Indicar cerrado/abierto ordenado
2	Resistencias de 220 ohmios	Limitar corriente de cada LED
1	Pulsador de cuatro patas	Bloquear mientras se mantiene pulsado
1 + 1	Módulo de alimentación y pila de 9 V con conector DC	Salida regulada a 5 V para el servo
-	Cartón, palitos opcionales y herramientas escolares	Base, soporte y barrera

Usa los cables Dupont del kit para conectar la placa y el sensor. No hace falta cortar UTP. La protoboard permite trabajar sin soldadura. Para sujetar el cartón puedes usar pestañas y ranuras; cinta o pegamento solo si ya los tienes.

Verificación al recibirlo

El archivo tutorial-ELEGO.pdf tiene 223 páginas y dice “The Most Complete Starter Kit”. Es una versión más amplia que el Super Starter comprado. Esta guía no depende de sus extras (RFID, teclado, adaptador de pared, etc.). Comprueba físicamente la lista anterior antes de armar.

Si falta la pila, el módulo o su conector, realiza primero el modo con LED por USB. No improvises una unión de pilas sueltas ni conectes 9 V al servo. Si falta un componente principal, habrá que adaptar esa parte al inventario real.

2. Preparar la computadora

Hazlo antes de llevar el proyecto al salón

1. Descarga Arduino IDE desde arduino.cc/en/software. Instálalo en tu computadora. Necesitas internet para esta preparación, no para ejecutar después el programa.

2. Abre el administrador de placas y comprueba que esté instalado **Arduino AVR Boards**. En el administrador de bibliotecas instala **Servo, de Arduino**, si no aparece disponible. El proyecto no necesita una biblioteca del sensor ultrasónico.

3. Conecta solo el UNO a la computadora con el USB del kit. Selecciona **Arduino Uno** y el puerto que aparece al conectarlo. No elijas Uno R4. En macOS suele comenzar por /dev/cu.; en Windows suele ser COM.

4. Abre Archivo > Ejemplos > 01.Basics > Blink. Pulsa Verificar y luego Subir. El LED integrado debe parpadear. Esto confirma comunicación y carga, todavía no el circuito externo.

5. Abre el archivo adjunto **barrera_ugm.ino**, dentro de la carpeta del mismo nombre. Alternativamente copia, en orden, todas las páginas de código del final en un sketch vacío y guárdalo como barrera_ugm. No copies los encabezados ni pies de página.

6. Para la primera etapa cambia **USAR_SERVO = true** por **USAR_SERVO = false**. Verifica y sube. Después abre el Monitor serie a **9600 baudios**. Sin sensor conectado verás distancia -1: significa lectura inválida.

Guardar para trabajar sin internet

Conserva localmente este PDF, el archivo .ino y el tutorial. Comprueba que Verificar funciona con internet desconectado. Deja la placa, el paquete AVR y la biblioteca Servo instalados. No necesitas cuenta de Arduino ni editor en línea.

El archivo .ino evita errores de copiar código desde el PDF. Cada vez que cambies algo, vuelve a pulsar Subir. Al reiniciar la placa o abrir el monitor puede reiniciarse el conteo a cero; no se guarda permanentemente.

3. Entender la protoboard

Todas las conexiones se hacen con USB y pila desconectados

En la zona central, cada grupo de cinco orificios de una misma fila está unido. La ranura central separa ambos lados: una fila a-e no está unida a f-j. Los rieles laterales distribuyen alimentación, pero pueden estar cortados a la mitad.



1. Sin alimentar nada, pon negro en COM y rojo en VΩmA del MUT-830. Selecciona continuidad. Junta las puntas para reconocer el sonido. Comprueba cinco orificios de una fila y después dos filas distintas.

2. Comprueba cada tramo de riel. Si está dividido, usa solo un tramo para cada alimentación o une sus mitades con un puente claramente identificado. Los colores impresos son etiquetas: no crean una conexión eléctrica.

3. Reserva la fila central 1 para **5 V de lógica**: une e1 con f1 para tener a1-j1 conectados. No pongas otro componente en esa fila. Los rieles laterales positivos serán solo **5 V del módulo para el servo**. GND será común. Etiqueta las zonas y nunca unas sus positivos.

4. Mide las resistencias fuera del circuito en la escala 2000 Ω. Busca aproximadamente 220 Ω. Código común de cuatro bandas: rojo-rojo-marrón-dorado; en cinco bandas: rojo-rojo-negro-negro-marrón. Usa la lectura para confirmar.

5. Identifica cada LED: pata larga, ánodo; lado plano y pata corta, cátodo. Si están recortadas, usa la prueba de diodo o revisa la pieza. Cada LED lleva su propia resistencia.

El pulsador

Sus cuatro patas forman dos pares unidos internamente. Con continuidad identifica un par de patas que **no** estén unidas en reposo y sí al pulsar. Usa una para D2 y otra para GND. No supongas la orientación por la apariencia. Así evitas que el bloqueo quede siempre activo.

4. Cablear la lógica

Primero sin servo y sin módulo de alimentación conectado

Origen	Destino	Comprobación
UNO 5V	Fila central 1 (5 V lógica)	Solo lógica y sensor
UNO GND	Riel negativo común	Referencia de todo el circuito
HC-SR04 VCC / GND	5 V lógica / GND	Lee las etiquetas del sensor
HC-SR04 TRIG / ECHO	UNO D7 / D8	No intercambiar
UNO D4	Resistencia 220 Ω → ánodo LED rojo	Cátodo rojo a GND
UNO D5	Resistencia 220 Ω → ánodo LED verde	Cátodo verde a GND
UNO D2	Una pata elegida del pulsador	La otra pata elegida a GND

Las flechas indican el camino eléctrico; una unión se hace usando la misma fila de cinco orificios o un cable. Las dos patas de una resistencia o de un LED deben quedar en filas eléctricamente distintas.

Qué hace el botón

El programa usa INPUT_PULLUP: una resistencia interna mantiene D2 en HIGH cuando está suelto. Al pulsar, D2 se une a GND y lee LOW. El programa traduce LOW como **bloqueo = 1**. No requiere una resistencia externa para el botón.

1. Revisa cada cable contra la tabla. Confirma que no hay un cable directo entre 5 V y GND. Conecta el USB y abre el monitor a 9600 baudios.

2. Apunta el sensor hacia una caja plana a unos 30 cm. Debe mostrar una distancia positiva, detectado=0, orden_abierta=0 y LED rojo encendido.

3. Acerca la caja a 5-8 cm, frente a ambos cilindros. Tras tres lecturas cercanas debe encenderse el verde y total aumentar una vez. Retira a 20-30 cm: tras confirmar ausencia y esperar 2 s debe volver el rojo.

El rojo significa “orden de barrera cerrada”, no exclusivamente “bloqueo”. El estado de bloqueo se ve en el monitor. No conectes todavía el servo si estas pruebas fallan.

5. Alimentar el servomotor

UNO por USB; servo por el módulo regulado del kit

El servo puede exigir más corriente al moverse que la lógica. Usaremos la pila de 9 V del kit conectada al **módulo regulador**, cuya salida debe ser 5 V. La pila rectangular puede agotarse pronto o caer de voltaje; no se garantiza una clase completa de movimiento.

1. Desconecta USB y pila. Coloca el módulo de alimentación en la protoboard verificando que sus terminales + y - coincidan con los rieles. Sitúa los selectores de salida en **5 V**, según la serigrafía. No lo pongas en rieles ya alimentados por el UNO.

2. El módulo puede alimentar ambos lados: comprueba qué rieles toca. Mantén el 5 V de lógica en la fila central 1 reservada, fuera de esos rieles. Nunca unas la salida positiva del módulo con UNO 5V.

3. Sin servo conectado, enchufa la pila con su conector DC al módulo y enciéndelo. MUT-830: negro COM, rojo VΩmA, DCV 20. Mide salida positiva respecto a negativa: debe estar cerca de **5 V** (aproximadamente 4.8-5.2 V). Si no, apaga y revisa el selector y rieles.

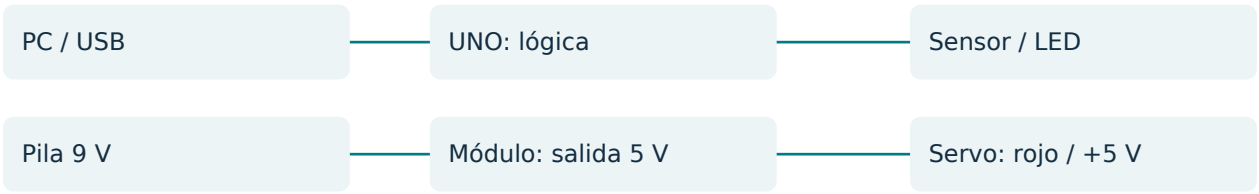
4. Apaga el módulo y retira la pila. Une **GND del módulo con GND del UNO**. Une rojo del servo a +5 V del módulo, marrón a GND y naranja (señal) a **D9**. Confirma las marcas del servo si sus colores difieren.

Unir	Mantener separados
GND UNO + GND módulo + GND sensor + marrón servo	5 V UNO y 5 V módulo
D9 UNO con señal naranja del servo	9 V de pila y cualquier pin del servo/UNO

5. Con el brazo plástico quitado, carga el programa con USAR_SERVO=true por USB. Enciende el módulo. Prueba unos pocos movimientos sin carga y observa el monitor. Para apagar: módulo primero y USB después; no recablees alimentado.

Si vibra, se reinicia o no se mueve: apaga, revisa conexiones y mide el 5 V del módulo durante el intento. Si cae o no sostiene el servo, usa USAR_SERVO=false y desconecta sus tres cables. La práctica de sensor, lógica, LED y conteo sigue funcionando sin compras. No lo soluciones aplicando 9 V directamente.

Mapa de alimentación



GND común en ambos circuitos. D9 envía señal al servo.

6. Construir y calibrar la barrera

Solo después de que funcione el servo sin carga

1. Haz una base de cartón de unos 25 × 20 cm. Dobla dos soportes con pestañas y ranuras: uno para el sensor y otro para el cuerpo del servo. Las dimensiones son orientativas. Mantén libres los conectores y el eje.

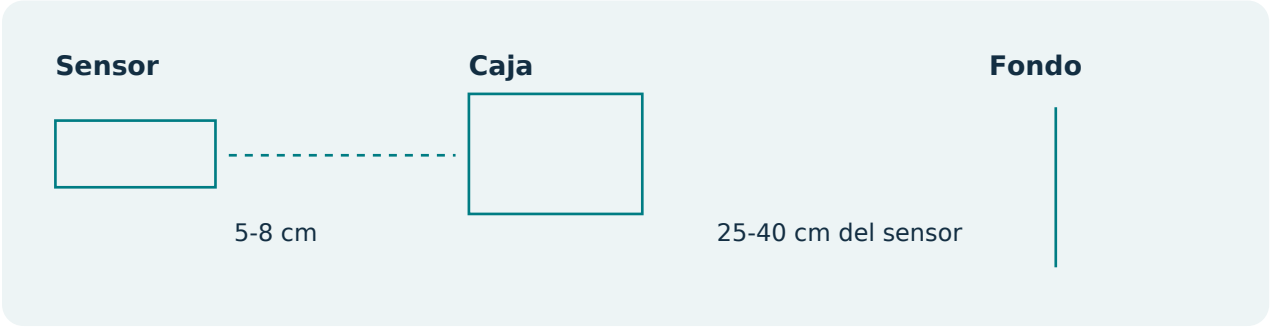
2. Corta una tira de cartón de unos 8 × 1 cm para el brazo. Es preferible al palito si este pesa más. Usa el brazo plástico incluido con el servo como unión. Sujeta la tira con pestañas, hilo o cinta que ya tengas, sin pegar el eje.

3. Deja la zona del sensor libre, con una pared o cartón de fondo a 25-40 cm. Con el programa encendido y sin objeto, el servo recibe CERRADO=20. Apaga módulo y USB. Coloca el brazo plástico horizontal sin girar el eje a la fuerza y fíjalo con el tornillo del kit.

4. Enciende y acerca la caja. ABIERTO=100 ordena un giro de 80 grados. Si baja en vez de subir, cambia la orientación del soporte o reajusta los ángulos con el brazo desmontado. Evita extremos 0/180 y cualquier tope físico.

5. Sitúa el sensor a un lado de la barrera, orientado hacia la zona donde presentarás la caja. El brazo no debe cruzar su campo de medida. Presenta la cara plana de una caja a 5-8 cm; deja el fondo a más de 15 cm.

Vista superior conceptual



La caja se presenta manualmente; no hay banda motorizada. Deja el giro libre, usa un brazo muy ligero y no detengas el servo con la mano. Si zumba de forma continua contra un tope, desconecta y corrige la geometría.

La regla de esta maqueta incluye memoria de posición y tiempo de espera. Es una ampliación de la regla instantánea de la banda de los MD, no su reproducción literal.

7. Entender y probar la regla

Tres lecturas confirman presencia; dos umbrales evitan cambios nerviosos

Presencia: tres lecturas consecutivas de 10 cm o menos. **Ausencia:** tres lecturas de 15 cm o más. Entre 10 y 15 cm se conserva el estado. El programa consulta aproximadamente cada 100 ms; una lectura de -1 es inválida.

Apertura: presencia confirmada y botón suelto. **Cierre:** barrera abierta, ausencia confirmada y distancia válida de 15 cm o más durante 2 s seguidos. **Bloqueo:** impide nuevas aperturas; no obliga a bajar sobre una caja. **Lectura inválida:** conserva la orden y cancela la espera de cierre.

Prueba, en este orden	Resultado esperado
1. Reiniciar con fondo a 30 cm	Rojo; total=0; orden_abierta=0
2. Acercar a 6 cm y mantener 5 s	Verde; total=1, no sigue sumando
3. Retirar a 30 cm y esperar 3 s	Rojo; total sigue en 1
4. Mantener botón y acercar otra vez	Sigue rojo; bloqueo=1; total=2
5. Soltar botón sin retirar caja	Abre; total sigue en 2
6. Pulsar con caja todavía presente	Permanece abierta: no es cierre forzado
7. Retirar caja a 30 cm	Cierra después de la espera
8. Cambiar distancia entre 11 y 14 cm	Conserva presencia previa; no inventa entradas

El contador aumenta en cada cambio confirmado de ausente a presente, incluso con bloqueo. Cuenta detecciones, no aperturas ni cajas transportadas. Al retirar y volver a acercar la misma caja se genera otra detección.

Prueba de lectura inválida

Con una barrera abierta, orienta el conjunto hacia una zona sin eco útil sin tocar conexiones. Cuando cm=-1, debe conservar orden_abierta=1. Restablece el fondo válido y despejado; entonces podrá cerrar tras la espera. Nunca desconectes cables para provocar fallas mientras esté energizado.

En el monitor: cm, detectado, bloqueo, orden_abierta, total. Por ejemplo **6.2,1,0,1,1**: objeto cercano, detectado, sin bloqueo, orden de abrir, una detección. No interpretes -1 como una distancia real.

8. Medir con el MUT-830

Primero demuestra tú; después rota las funciones entre alumnos

A. Continuidad y resistencia, sin energía

Retira USB y pila. Usa COM y VΩmA. En continuidad identifica rieles, filas y pulsador. Para medir una resistencia de 220 Ω, sácala del circuito o levanta una pata; selecciona 2000 Ω. Una lectura “1” fuera de rango no equivale a cero. No pruebes continuidad en pilas ni con el circuito alimentado.

B. Voltaje, sin abrir el circuito

Con rojo en VΩmA, negro en COM y DCV 20, mide 5 V lógica respecto a GND y 5 V servo respecto a GND. Mide también entre las patas de la resistencia del LED encendido. Coloca las puntas sin puentear filas vecinas. Anota valores reales.

Ejemplo, no valor obligatorio: si en la resistencia de 220 Ω mides 3.0 V, I=3.0/220=0.0136 A=13.6 mA. Su potencia es P=3.0×0.0136=0.041 W. El voltaje restante cae principalmente en el LED. No uses 5/220 como corriente exacta del LED.

C. Corriente del LED, abriendo su rama

1. Elige una rama LED con su resistencia de 220 Ω comprobada. Desconecta USB y pila. Retira la conexión entre D4 (rojo) y su resistencia. Ese hueco es donde irá el multímetro.

2. Negro en COM y rojo en VΩmA. Selecciona **DCA 200m**. Punta roja a D4; punta negra al extremo libre de la resistencia. El resto de esa rama sigue: resistencia, LED y GND. Así el medidor está en serie.

3. Enciende por USB en un estado con LED rojo activo. Lee mA. Para esta rama limitada por 220 Ω a 5 V la corriente es menor de 23 mA. No uses este procedimiento para el servo ni para toda la alimentación.

4. Apaga y desconecta. Retira el instrumento, restituye el cable de D4 y coloca el selector en DCV 20 antes de volver a medir voltaje. Nunca pongas las puntas en paralelo con una fuente mientras esté en DCA.

El borne VΩmA admite hasta 200 mA de medición; el fusible no aumenta ese límite. El rango de 10 A carece de fusible según el manual: no lo usamos en este proyecto. Tampoco deducimos la corriente del servo midiendo su resistencia.

9. Actividad para el grupo

Una sola maqueta; todos predicen, observan y explican

Trabaja con roles rotativos: lector del esquema, montador, responsable del multímetro, operador de caja/botón y relator. Con cinco alumnos, uno por rol; con más, equipos de dos o tres. Solo una persona recablea y siempre sin alimentación.

Momento	Tiempo orientativo	Trabajo
Antes de la llegada	Dentro de módulos 1-2	Pilas, UTP, LED y resistencias. Dibujar entradas y salidas.
Preparación docente	60-90 min o más si hay fallas	Instalar software, revisar piezas y probar sin carga.
Sesión de montaje	90 min	Lógica y sensor (40), alimentación y servo (25), cartón y ajuste (25).
Sesión de aplicación	60 min	Pruebas (20), mediciones (20), explicaciones y cambios (20).

Los tiempos de aula suponen software preparado y montaje probado por el profesor. Puedes conservar el circuito entre sábados para no gastar tiempo reconstruyéndolo. No requiere tarea extensa ni define una calificación.

Registro breve en el cuaderno

Medición / situación	Predicción	Dato / explicación
Resistencia elegida (Ω)		
Voltaje en resistencia del LED (V)		
I calculada / I medida (mA)		
Caja quieta durante 5 s		
Caja presente y bloqueo pulsado		
Distancia inválida con barrera abierta		

Reto: cambia ESPERA_MS de 2000 a 4000, vuelve a subir y comprueba el cambio. Después explica por qué aumentar el tiempo no aumenta el voltaje. Reto adicional: propone qué sensor faltaría para confirmar que la caja llegó al otro lado, sin comprarlo ni montarlo.

10. Respuestas y fallas frecuentes

Para el profesor

Pregunta	Explicación esperada
¿Por qué cada LED lleva resistencia?	Limita la corriente. El cable no reemplaza esa función.
¿Por qué no sigue contando una caja quieta?	Solo cuenta el cambio confirmado de ausencia a presencia.
¿Puede contar estando bloqueado?	Sí. Detección y permiso de apertura son funciones distintas.
¿Verde demuestra movimiento?	No. Muestra la orden; puede fallar alimentación o mecánica.
¿Arduino alimenta el motor por D9?	No. D9 envía la señal. El módulo suministra energía al servo.
¿Por qué comparar I medida y calculada?	Relaciona Ohm con una resistencia real; tolerancias y el medidor explican pequeñas diferencias.

Diagnóstico por síntomas

Síntoma	Revisar, apagando antes de recablear
No aparece puerto	Cable USB de datos, conector y selección Arduino Uno. Probar Blink solo.
Servo.h no encontrado	Instalar biblioteca Servo de Arduino antes de clase.
Distancia siempre -1	5 V, GND, TRIG D7, ECHO D8; caja plana a 5-30 cm y fondo.
LED apagado	Polaridad, resistencia y filas. Probar el estado que lo enciende.
Bloqueo siempre 1	Patas elegidas del botón: quizá uniste un par permanentemente conectado.
Servo vibra o la placa reinicia	GND común, alimentación separada, pila y carga. Pasar a modo LED si no es estable.
Cierra o cuenta inesperadamente	Brazo fuera del campo del sensor, fondo estable y posición de caja. Revisar cm en monitor.

11. Antes de la clase y fuentes

Comprobaciones finales y relación con el material existente

Comprueba: Blink funciona; modo LED supera las pruebas; salida del módulo cerca de 5 V; positivos separados y GND común; servo sin carga estable; brazo libre; conteo no repetido; botón correcto; monitor a 9600; programa y PDF disponibles sin internet.

Si el servo no funciona con la alimentación disponible, conserva la misma lógica con USAR_SERVO=false y retira sus tres cables. El verde representa apertura y el rojo cierre. Los alumnos pueden mover un brazo manualmente siguiendo la orden para identificar la función del actuador. Esta alternativa no valida el movimiento eléctrico del motor.

Fuentes de preparación

Tutorial aportado: tutorial-ELEGO.pdf, “The Most Complete Starter Kit Tutorial for UNO”, V1.0.19.01.23. Inventario: pp. 4-9. IDE y carga: pp. 12-42. LED: pp. 43-49. Entradas: pp. 59-63. Servo: pp. 76-79. Ultrasónico: pp. 80-84. Consulta las imágenes de esas lecciones para reconocer piezas, pero usa los pines de esta guía.

La guía combina esos conceptos con código propio. No copia la conexión de alimentación del servo del ejemplo inicial: aquí se separa del UNO. El manual amplio no demuestra por sí solo qué extras llegarán en la caja.

Arduino: selección de placa y puerto, biblioteca Servo y solución de problemas de alimentación.
Selección de placa y puerto
Alimentación y problemas de servomotores
Descarga de Arduino IDE

TRUPER: instructivo MUT-830, código 10400, secciones de tensión, corriente y continuidad.
Manual oficial MUT-830

Archivo de programa

El archivo complementario está en material/proyecto/barrera_ugm/barrera_ugm.ino. Contiene exactamente el código de las páginas siguientes. Conserva esa estructura de carpeta para abrirlo en Arduino IDE.

Las pruebas automáticas comprueban las decisiones de apertura, bloqueo, cierre, conteo y lectura inválida. No sustituyen una carga real en UNO ni la prueba del sensor, servo, pila y módulo cuando llegue el kit.

Programa completo · 1/3

Copia los bloques en orden dentro del mismo sketch; mejor abre el archivo .ino

```
#include <Servo.h>
// UGM Playa Vicente - Tirso Reyes
// Maqueta didactica, no control de acceso real.
const byte TRIG = 7, ECHO = 8, SERVO = 9;
const byte ROJO = 4, VERDE = 5, BLOQUEO = 2;
const bool USAR_SERVO = true; // false: solo indicadores
const int CERRADO = 20, ABIERTO = 100;
const float ENTRADA_CM = 10, SALIDA_CM = 15;
const unsigned long ESPERA_MS = 2000;
Servo brazo;
bool presente = false, abierta = false;
byte cerca = 0, lejos = 0;
unsigned long contador = 0, inicioLibre = 0;
bool esperando = false;

float distanciaCm() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  unsigned long t = pulseIn(ECHO, HIGH, 25000UL);
  if (t == 0) return -1;
  float cm = t / 58.0;
  if (cm < 2 || cm > 200) return -1;
  return cm;
}

void aplicar(bool orden) {
  if (orden != abierta) {
    abierta = orden;
    if (USAR_SERVO)
      brazo.write(abierta ? ABIERTO : CERRADO);
  }
  digitalWrite(VERDE, abierta ? HIGH : LOW);
  digitalWrite(ROJO, abierta ? LOW : HIGH);
}

void actualizar(float cm, bool bloqueo, unsigned long ahora) {
  if (cm < 0) {
    cerca = lejos = 0;
    esperando = false;
    // Lectura invalida: conservar posicion; no ordenar cierre.
```

Programa completo · 2/3

Copia los bloques en orden dentro del mismo sketch; mejor abre el archivo .ino

```
    return;
  }
  bool anterior = presente;
  if (cm <= ENTRADA_CM) {
    lejos = 0;
    if (cerca < 3) cerca++;
    if (cerca == 3) presente = true;
  } else if (cm >= SALIDA_CM) {
    cerca = 0;
    if (lejos < 3) lejos++;
    if (lejos == 3) presente = false;
  } else {
    cerca = lejos = 0; // zona intermedia: conservar estado
  }
  if (presente && !anterior) contador++;
  if (presente) {
    esperando = false;
    if (!bloqueo) aplicar(true);
  } else if (abierta) {
    // Cierre solo con distancia claramente libre y valida.
    if (cm >= SALIDA_CM) {
      if (!esperando) {
        esperando = true;
        inicioLibre = ahora;
      }
      if (ahora - inicioLibre >= ESPERA_MS) aplicar(false);
    } else {
      esperando = false;
    }
  }
}

void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  pinMode(ROJO, OUTPUT);
  pinMode(VERDE, OUTPUT);
  pinMode(BLOQUEO, INPUT_PULLUP);
  digitalWrite(TRIG, LOW);
  digitalWrite(ROJO, HIGH);
  digitalWrite(VERDE, LOW);
  if (USAR_SERVO) {
    brazo.attach(SERVO);
```

Programa completo · 3/3

Copia los bloques en orden dentro del mismo sketch; mejor abre el archivo .ino

```
    brazo.write(CERRADO);
  }
  Serial.begin(9600);
  Serial.println("cm,detectado,bloqueo,orden_abierta,total");
}

void loop() {
  float cm = distanciaCm();
  bool bloqueo = digitalRead(BLOQUEO) == LOW;
  actualizar(cm, bloqueo, millis());
  Serial.print(cm); Serial.print(',');
  Serial.print(presente); Serial.print(',');
  Serial.print(bloqueo); Serial.print(',');
  Serial.print(abierta); Serial.print(',');
  Serial.println(contador);
  delay(100);
}
```